Questini Technical Report

Mohamed Belgacem

May 6, 2026

Contents

1 Architecture Components

The project follows a layered structure that separates presentation, state management, and data access responsibilities.

The presentation layer is implemented with Jetpack Compose screens and reusable UI components. The state layer is centered on a ViewModel that exposes observable state to the UI and coordinates quiz flow. The data layer is based on a repository plus an asset loader that reads question data from JSON.

This split improves maintainability and allows independent evolution of UI, business rules, and data sources.

2 Navigation Component

Navigation is handled through the Jetpack Navigation Component for Compose. Routes are centralized and consumed through a single application navigation graph.

The current quiz journey is structured as a guided flow:

- Splash to Main Menu
- Main Menu to Category Selection
- Category Selection to Difficulty Selection
- Difficulty Selection to Quiz
- Quiz to Results

This route design provides predictable transitions and makes flow updates easier to implement and test.

3 Activity Lifecycle

The application uses lifecycle-aware architecture principles. The Activity acts as a host for Compose content and delegates state ownership to the ViewModel.

Lifecycle-sensitive tasks such as timers and long-running state updates are managed in a way that respects lifecycle boundaries. This reduces the risk of leaks, stale state, and invalid UI updates during configuration changes or background transitions.

4 Testing Coverage

Testing coverage now includes multiple layers:

- Unit tests for repository filtering and route consistency
- Modular tests for data loading behavior and question set validation
- Navigation-focused tests for route behavior and flow integrity

Local JVM tests compile and pass. Instrumentation tests compile successfully, while execution on physical devices depends on device install permissions and USB policy settings.

5 Adaptive UI

The UI is Compose-based and responsive to a range of device dimensions. Layouts use flexible spacing and scalable components rather than fixed-position assumptions.

Content cards, bottom navigation, and quiz surfaces are structured to remain usable on both compact and larger screens. This supports progressive enhancement without requiring separate screen implementations.

6 UI/UX

The interface design aims for a clear and modern educational experience with a consistent visual identity.

Key UX characteristics include:

- Clear hierarchy in onboarding and quiz progression
- Focused decision points (category then difficulty)
- Readable typography and contrast-aware color usage
- Lightweight motion to support orientation without distraction

The result is a flow that is easy to understand and quick to use.

7 Functionality

Core functionality currently implemented includes:

- Quiz initialization from selected category and difficulty
- Asset-backed question loading and filtering
- Answer selection, scoring, and result generation
- Navigation among menu, category, difficulty, quiz, and results

- Launcher branding and theme consistency

Recent content updates added a full Cities dataset and corresponding drawable integration.

8 Code Quality

Code quality is supported through separation of concerns, readable file organization, and incremental test coverage.

Current strengths include:

- Clear package boundaries for data, navigation, UI, and state
- Reusable screen components and centralized route definitions
- Compile-time verification through Gradle build tasks
- Regression protection via targeted tests

Recommended next steps are broader instrumentation execution on unrestricted devices, stronger negative-case data tests, and continued refinement of route-level test depth.

9 Conclusion

Questini now has a solid and organized foundation suitable for iterative production development. The architecture, navigation flow, and test strategy align with modern Android engineering practices while remaining practical for fast feature delivery.